Day 19 — Deploying ML and GenAI Models with REST APIs and Web Applications

5-line beginner summary

1. A trained model becomes useful when other applications can send it data and receive predictions through an API.
2. FastAPI is a Python framework commonly used to expose ML, RAG, and agent workflows as REST endpoints.
3. Production APIs require validation, authentication, rate limiting, logging, monitoring, and reliable error handling.
4. Short predictions can run synchronously, while long document, RAG, and agent operations should often use asynchronous jobs.
5. The model is only one component; a production AI service also needs security, versioning, scalability, observability, and governance.

---

1. Why REST APIs are used for AI services

A machine learning model normally starts as a Python object:

```
prediction = model.predict(customer_data)
```

This works inside the developer's notebook, but another application cannot directly access that Python object.

A REST API creates a network-accessible interface:

```
Application sends JSON request
            ↓
API validates request
            ↓
API calls model
            ↓
API returns JSON response
```

For example, a banking application could send:

```
{
  "age": 38,
  "income": 85000,
  "loan_amount": 300000
}
```

The API may return:

```
{
  "prediction": "low_risk",
  "probability": 0.87,
  "model_version": "3.2"
}
```

REST APIs are useful because the client and model service can use different technologies:

```
React web application
Java banking application
Mobile application
Python batch process
Power BI dashboard
        │
        └── All can call the same REST API
```

FastAPI is a Python framework for building APIs using Python type hints, and it automatically integrates request validation and API documentation. (FastAPI)

---

2. Request and response design

A good API contract clearly defines:

- What the client must send.
- What the server returns.
- Which fields are mandatory.
- Which data types are allowed.
- How errors are represented.
- Which model and API version produced the answer.

Example ML request

```
POST /v1/predictions
Content-Type: application/json
Authorization: Bearer <token>
```

```
{
  "customer_id": "C10245",
  "features": {
    "age": 38,
    "annual_income": 85000,
    "loan_amount": 300000,
    "credit_score": 745
  }
}
```

Example ML response

```
{
  "request_id": "req-8f72a",
  "prediction": "approved",
  "probability": 0.91,
  "model_name": "loan-risk-model",
  "model_version": "3.2",
  "processing_time_ms": 47
}
```

Recommended response fields

Field	Purpose
`request_id`	Trace the request across services
`prediction` or `answer`	Main model result
`confidence`	Model probability when meaningful
`model_version`	Reproducibility and auditing
`processing_time_ms`	Performance debugging
`citations`	Sources used by a RAG system
`warnings`	Low confidence or incomplete data
`usage`	Token or compute consumption

Do not expose internal stack traces, database details, prompt templates, secrets, or infrastructure names in the response.

FastAPI response models can validate, document, convert, and filter outgoing data so that accidental internal fields are not returned. (FastAPI)

---

3. FastAPI basics

A simple FastAPI application looks like this:

```
from fastapi import FastAPI

app = FastAPI()

@app.get("/health")
async def health():
    return {"status": "healthy"}
```

Run it using an ASGI server such as Uvicorn:

```
uvicorn main:app --host 0.0.0.0 --port 8000
```

FastAPI is an ASGI framework, and an ASGI server such as Uvicorn handles incoming network requests. (FastAPI)

Basic prediction endpoint

```
@app.post("/v1/predict")
def predict(request: PredictionRequest):
    result = model.predict(request.features)
    return {"prediction": result}
```

FastAPI also generates interactive OpenAPI documentation, normally available during development through endpoints such as `/docs`.

Recommended project structure

```
ai-service/
│
├── app/
│   ├── main.py
│   ├── api/
│   │   ├── prediction_routes.py
│   │   ├── rag_routes.py
│   │   └── agent_routes.py
│   ├── schemas/
│   │   ├── requests.py
│   │   └── responses.py
│   ├── services/
│   │   ├── model_service.py
│   │   ├── rag_service.py
│   │   └── agent_service.py
│   ├── security/
│   │   └── authentication.py
│   └── observability/
│       ├── logging.py
│       └── metrics.py
│
├── models/
├── tests/
├── Dockerfile
├── requirements.txt
└── README.md
```

Keep API routing, business logic, model logic, and infrastructure code separate.

---

4. Input validation

Never send unvalidated client data directly to the model.

Validation protects the application from:

- Missing fields.
- Incorrect data types.
- Impossible values.
- Oversized text.
- Unsupported file formats.
- Prompt injection payloads.
- Unexpected JSON fields.

FastAPI uses Pydantic models to parse and validate request bodies. Invalid input produces a structured validation error before the endpoint logic runs. (FastAPI)

Example validation model

```
from pydantic import BaseModel, Field
```

```
class LoanRequest(BaseModel):
    age: int = Field(ge=18, le=100)
    annual_income: float = Field(gt=0, le=100_000_000)
    loan_amount: float = Field(gt=0)
    credit_score: int = Field(ge=300, le=900)
```

This prevents invalid requests such as:

```
{
  "age": -15,
  "annual_income": "unknown",
  "loan_amount": -50000,
  "credit_score": 1500
}
```

GenAI validation examples

For an LLM or RAG request, validate:

```
class RAGRequest(BaseModel):
    question: str = Field(min_length=3, max_length=2000)
    conversation_id: str | None = None
    top_k: int = Field(default=5, ge=1, le=20)
```

Also validate business rules:

```
Is the user permitted to access this document collection?
Is the requested model allowed for this department?
Is the input document within the size limit?
Is the selected language supported?
```

Pydantic field validators can implement additional checks or controlled transformations beyond basic type validation. (Pydantic)

---

5. Error handling

Errors should be predictable and machine-readable.

Standard error response

```
{
  "error": {
    "code": "INVALID_INPUT",
    "message": "credit_score must be between 300 and 900",
    "request_id": "req-8f72a"
  }
}
```

Common status codes

Status	Meaning	AI example
200	Successful request	Prediction completed

Status	Meaning	AI example
201	Resource created	Agent session created
202	Accepted for processing	Long-running job submitted
400	Invalid business request	Unsupported model parameter
401	Authentication missing/invalid	Invalid token
403	Authenticated but not permitted	User cannot access HR documents
404	Resource not found	Job or model not found
409	Conflict	Duplicate request ID
422	Request validation failed	Wrong field type
429	Too many requests	Rate limit exceeded
500	Unexpected server failure	Internal model error
503	Service temporarily unavailable	Model server unavailable
504	Downstream timeout	LLM provider timed out

HTTP uses the `2xx` class for successfully received, understood, and accepted requests. (RFC Editor)

FastAPI provides `HTTPException` for controlled client-facing errors and allows custom handlers for request validation and application exceptions. (FastAPI)

Example

```
@app.post("/v1/predict")
def predict(request: LoanRequest):
    try:
        prediction = model_service.predict(request)
        return prediction

    except ModelUnavailableError:
        raise HTTPException(
            status_code=503,
            detail={
                "code": "MODEL_UNAVAILABLE",
                "message": "Prediction service is temporarily unavailable"
            }
        )

    except Exception:
        logger.exception("Unexpected prediction failure")

        raise HTTPException(
            status_code=500,
            detail={
                "code": "INTERNAL_ERROR",
                "message": "The request could not be completed"
```

```
            }
        )
```

Log the technical error internally, but return a safe error message to the client.

---

6. Authentication basics

Authentication answers:

> Who is calling the API?

Authorization answers:

> What is that caller allowed to do?

Common authentication methods

API key

Suitable for simple service-to-service access:

```
X-API-Key: abc123
```

API keys should be stored in a secret manager, rotated regularly, hashed where appropriate, and never committed to Git.

Bearer token or JWT

Suitable for user-facing applications:

```
Authorization: Bearer eyJhbGciOi...
```

The API validates:

- Token signature.
- Expiration time.
- Issuer.
- Audience.
- User identity.
- Roles or scopes.

FastAPI supports standard security dependencies, including OAuth2 bearer-token patterns and JWT-based implementations. (FastAPI)

Example authorization

```
@app.post("/v1/hr-rag/query")
def query_hr_documents(
    request: RAGRequest,
    user: User = Depends(require_authenticated_user)
):
    if "hr-policy-reader" not in user.roles:
        raise HTTPException(status_code=403, detail="Access denied")
```

```
    return rag_service.answer(
        question=request.question,
        user_id=user.id,
        allowed_departments=user.departments
    )
```

For RAG, authorization must also be applied during retrieval. It is not enough to protect only the endpoint.

```
Wrong:
User passes API authentication
        ↓
Retriever searches every company document

Correct:
User passes API authentication
        ↓
Retriever filters documents using user permissions
```

Use HTTPS for production APIs because credentials, tokens, prompts, and predictions must be protected while travelling across the network. OWASP recommends HTTPS-only REST services. (OWASP Cheat Sheet Series)

---

7. Rate limiting

Rate limiting controls how frequently a client can call the API.

Example policy:

```
Normal ML API:
100 requests per minute per API key

RAG API:
30 requests per minute per user

Expensive agent API:
5 concurrent agent runs per user

Authentication endpoint:
3 failed login attempts per minute
```

Rate limiting protects against:

- Denial-of-service attacks.
- Accidental request loops.
- Excessive cloud costs.
- GPU overload.
- LLM token abuse.
- Brute-force authentication attempts.

OWASP identifies unrestricted resource consumption as a major API security risk and recommends rate limits that are tuned to individual business operations. (OWASP Foundation)

GenAI-specific limits

Consider limiting:

- Requests per minute.
- Concurrent requests.
- Input tokens.
- Output tokens.
- Document size.
- Agent execution steps.
- Tool calls per run.
- Total cost per user or department.

Example response:

```
{
  "error": {
    "code": "RATE_LIMIT_EXCEEDED",
    "message": "You have exceeded 30 RAG requests per minute",
    "retry_after_seconds": 22
  }
}
```

---

8. Logging

Logging records important events produced by the service.

Useful structured log

```
{
  "timestamp": "2026-07-11T12:15:24+05:30",
  "level": "INFO",
  "service": "policy-rag-api",
  "request_id": "req-8f72a",
  "endpoint": "/v1/rag/query",
  "user_id_hash": "usr-93c2",
  "model": "enterprise-llm",
  "model_version": "2026-06",
  "retrieved_chunks": 5,
  "latency_ms": 1240,
  "input_tokens": 480,
  "output_tokens": 212,
  "status_code": 200
}
```

Log these

- Request ID.

- Endpoint.
- Status code.
- Response time.
- Model and prompt version.
- Retrieval duration.
- Number of retrieved chunks.
- Input and output token counts.
- Tool calls.
- Retry counts.
- Model errors.
- Authorization failures.
- Safety-filter decisions.

Do not log these by default

- Passwords.
- Access tokens.
- API keys.
- Full personal data.
- Confidential documents.
- Entire prompts containing private data.
- Raw embeddings.
- Sensitive model responses.

Use structured JSON logs rather than long unstructured sentences because they are easier to search and aggregate.

---

9. Observability

Logging tells you that an event happened.

Observability helps you understand why the system is behaving in a certain way.

The main observability signals are:

```
Logs    → individual events
Metrics → numerical measurements over time
Traces  → path of one request across services
```

OpenTelemetry defines traces, metrics, and logs as core telemetry signals and provides vendor-neutral instrumentation for collecting and exporting them. (OpenTelemetry)

Infrastructure metrics

```
CPU usage
Memory usage
GPU utilization
GPU memory
Container restarts
```

```
Request count
Error rate
p50 latency
p95 latency
p99 latency
Queue length
```

ML metrics

```
Prediction distribution
Feature distribution
Missing-feature rate
Confidence distribution
Model drift
Business outcome
Accuracy after labels arrive
```

RAG metrics

```
Retrieval latency
LLM generation latency
Context precision
Context recall
Groundedness
Citation coverage
No-answer rate
Hallucination rate
```

Agent metrics

```
Agent completion rate
Average steps per run
Tool-call success rate
Approval rate
Agent timeout rate
Cost per run
Repeated-action rate
```

Trace example

```
Request: req-8f72a
│
├── Authentication.............. 12 ms
├── Permission lookup........... 25 ms
├── Query embedding............. 48 ms
├── Vector search............... 95 ms
├── Reranking................... 130 ms
├── Prompt construction.......... 10 ms
└── LLM generation.............. 910 ms

Total......................... 1230 ms
```

Without tracing, you only know that the request took 1.23 seconds. With tracing, you know that LLM generation used most of the time.

---

10. Async processing

There are two meanings of asynchronous processing that are often confused.

A. Async web programming

FastAPI supports `async def`, which is useful when the endpoint waits for I/O operations such as:

- Database queries.
- Vector database searches.
- Cloud LLM requests.
- HTTP tool calls.
- Object storage operations.

FastAPI documents `async` and `await` for concurrent I/O-based work. (FastAPI)

```
@app.post("/v1/rag/query")
async def query(request: RAGRequest):
    documents = await vector_database.search(request.question)
    answer = await llm_client.generate(request.question, documents)
    return answer
```

However:

```
async def does not automatically make CPU-heavy model inference faster.
```

CPU- or GPU-heavy work may still require:

- Multiple processes.
- A dedicated inference server.
- Batching.
- A task queue.
- GPU scheduling.

B. Long-running job processing

An agent may require several minutes because it:

- Searches documents.
- Calls several tools.
- Waits for human approval.
- Generates a report.
- Processes a large document.

Do not keep the HTTP request open indefinitely.

Job submission

```
POST /v1/agent-runs
```

Response:

```
{
  "job_id": "job-7482",
  "status": "queued",
  "status_url": "/v1/agent-runs/job-7482"
}
```

Job status

```
GET /v1/agent-runs/job-7482
```

```
{
  "job_id": "job-7482",
  "status": "running",
  "progress": 60
}
```

Completed result

```
{
  "job_id": "job-7482",
  "status": "completed",
  "result": {
    "report_url": "/v1/reports/report-782"
  }
}
```

FastAPI supports background tasks that execute after the response has been sent. (FastAPI)

For important, heavy, retryable, or long-running jobs, use an external queue:

```
API
  │
  ├── Add job to queue
  │
  └── Return job ID
          ↓
     Task queue
          ↓
     Worker service
          ↓
   Model / agent execution
          ↓
     Result database
```

Typical components include Redis or a message broker, worker processes, a result database, and retry/dead-letter handling.

---

11. Model serving

Model serving means making a trained model available for inference.

Pattern 1: Load the model inside FastAPI

```
FastAPI process
├── Authentication
├── Validation
├── Business logic
└── ML model in memory
```

Suitable for:

- Small Scikit-learn models.
- Small NLP models.
- Low-traffic internal applications.
- Prototypes and POCs.

Load the model once when the application starts, not for every request.

FastAPI lifespan events can load expensive model resources before the application starts receiving traffic and release them during shutdown. (FastAPI)

```
@asynccontextmanager
async def lifespan(app):
    app.state.model = load_model("models/model.pkl")
    yield
    unload_model(app.state.model)

app = FastAPI(lifespan=lifespan)
```

Pattern 2: Separate API and model server

```
Client
  ↓
FastAPI gateway
  ↓
Dedicated model server
  ↓
CPU/GPU model
```

Suitable for:

- Large deep-learning models.
- GPU inference.
- Multiple model versions.
- High traffic.
- Dynamic batching.
- Independent model scaling.

The FastAPI layer handles:

```
Authentication
Authorization
Validation
Rate limiting
Business rules
Request transformation
Response formatting
```

The model server handles:

```
Model loading
GPU execution
Batching
Inference optimization
Model replica scaling
```

MLflow can package models and expose them as REST inference endpoints locally, in containers, or through cloud platforms. (MLflow AI Platform)

Health endpoints

```
GET /health/live
GET /health/ready
```

Example:

```
{
  "status": "ready",
  "model_loaded": true,
  "model_version": "3.2",
  "vector_database": "connected"
}
```

- **Liveness:** Is the process alive?
- **Readiness:** Can it safely receive traffic?

Important scaling caution

Multiple server workers can improve request concurrency and use multiple CPU cores, but each process may load its own model copy. A four-worker service could therefore consume approximately four model copies in memory. FastAPI supports multiple Uvicorn worker processes, but worker count should be chosen according to model size and available resources. (FastAPI)

---

12. RAG API design

A RAG API usually performs:

```
Question
   ↓
Authentication and document permissions
   ↓
```

```
Query preprocessing
   ↓
Embedding
   ↓
Vector or hybrid search
   ↓
Reranking
   ↓
Prompt construction
   ↓
LLM generation
   ↓
Answer with citations
```

Endpoint

```
POST /v1/rag/query
```

Request

```
{
  "question": "How many annual leave days do employees receive?",
  "conversation_id": "conv-728",
  "filters": {
    "country": "India",
    "department": "Engineering"
  }
}
```

Response

```
{
  "request_id": "req-927a",
  "answer": "Employees in India receive 24 days of annual leave.",
  "citations": [
    {
      "document_id": "hr-policy-2026",
      "title": "India Leave Policy",
      "page": 8,
      "chunk_id": "chunk-82"
    }
  ],
  "grounded": true,
  "model_version": "enterprise-llm-2026-06",
  "retriever_version": "hybrid-reranker-v4"
}
```

RAG API design rules

Apply authorization during retrieval

```
documents = retriever.search(
    query=request.question,
```

```
    filters={
        "allowed_groups": current_user.groups,
        "country": request.filters.country
    }
)
```

Return citations

A RAG answer without sources is difficult to verify and audit.

Support “I do not know”

```
{
  "answer": null,
  "status": "insufficient_evidence",
  "message": "The available documents do not contain enough information."
}
```

Do not allow unrestricted retrieval parameters

A public client should not freely set:

```
top_k = 10,000
disable_permissions = true
return_full_documents = true
```

Server-side policies should enforce safe limits.

Track component versions

Record:

```
Embedding model version
Chunking version
Retriever version
Reranker version
Prompt version
LLM version
Knowledge-base version
```

This makes RAG responses reproducible and auditable.

---

13. Agent API design

An agent API is more complex than a normal prediction API because an agent may:

- Maintain state.
- Plan several steps.
- Call external tools.
- Modify systems.
- Pause for approval.
- Continue after human feedback.

Agent submission request

```
POST /v1/agent-runs
```

```
{
  "agent": "incident-analysis-agent",
  "task": "Investigate the latest payment-service failure",
  "session_id": "session-782",
  "max_steps": 10,
  "approval_mode": "required_for_write_actions"
}
```

Response

```
{
  "run_id": "run-9281",
  "status": "queued",
  "created_at": "2026-07-11T12:30:00+05:30"
}
```

Agent run state

```
QUEUED
  ↓
RUNNING
  ↓
WAITING_FOR_APPROVAL
  ↓
RUNNING
  ↓
COMPLETED / FAILED / CANCELLED
```

Approval request

```
{
  "run_id": "run-9281",
  "status": "waiting_for_approval",
  "proposed_action": {
    "tool": "restart_service",
    "target": "payment-service-production",
    "reason": "Service health checks are failing"
  }
}
```

Approval endpoint

```
POST /v1/agent-runs/run-9281/approvals
```

```
{
  "decision": "approved",
  "comment": "Proceed during the maintenance window"
}
```

Essential agent safeguards

Maximum execution limits

```
Maximum steps
Maximum execution time
Maximum token budget
Maximum tool calls
Maximum financial cost
```

Tool allowlist

```
Allowed:
- Read logs
- Search documentation
- Query monitoring data

Approval required:
- Restart service
- Update ticket
- Send email
- Modify database

Prohibited:
- Delete production database
- Export confidential documents
```

Idempotency

A repeated request should not accidentally perform the same write action twice.

```
Idempotency-Key: agent-action-7281
```

Audit trail

Record:

```
User request
Agent plan
Retrieved evidence
Model version
Tool arguments
Tool results
Approval decision
Final answer
Errors and retries
```

---

14. API versioning

AI systems change frequently:

- Feature names change.
- Model inputs change.

- Response structure changes.
- New citation fields are added.
- Agent states are added.
- Old models are retired.

Version the external API separately from the model.

```
API version:   v1
Model version: loan-risk-3.2
Prompt version: rag-policy-7
Retriever version: hybrid-4
```

URL versioning

```
POST /v1/predictions
POST /v2/predictions
```

This is simple and visible.

Safe change

Adding an optional field is usually less disruptive:

```
{
  "prediction": "approved",
  "confidence": 0.91
}
```

Breaking change

Changing:

```
{
  "prediction": "approved"
}
```

to:

```
{
  "result": {
    "decision": "approved"
  }
}
```

may break existing clients.

Recommended process

```
Create v2
   ↓
Run v1 and v2 together
   ↓
Notify consumers
   ↓
```

```
Measure v1 usage
   ↓
Set deprecation date
   ↓
Remove v1 after migration
```

Never silently replace a production model or API contract without traceability.

---

15. Production readiness checklist

API contract

- Request and response schemas are documented.
- Required and optional fields are clear.
- Maximum payload sizes are enforced.
- Errors use a consistent structure.
- API and model versions are returned.
- OpenAPI documentation is reviewed.

Model

- Model loads during startup.
- Model artifact checksum or version is recorded.
- Input schema matches the training schema.
- Prediction output is validated.
- Warm-up inference is performed.
- Fallback or rollback model is available.
- Model dependencies are pinned.

RAG

- Document-level authorization is implemented.
- Citations are returned.
- Retrieval filters are validated.
- Prompt injection defenses are present.
- No-answer behaviour is defined.
- Embedding, chunking, reranking, prompt, and LLM versions are tracked.

Agent

- Tool allowlists are defined.
- Write actions require approval where needed.
- Maximum steps and costs are enforced.
- Duplicate actions are prevented.
- Cancellation is supported.
- Every action is auditable.

Security

- HTTPS is enabled.
- Authentication is enforced.
- Authorization is tested.
- Secrets are stored outside source code.
- Tokens expire and can be rotated.
- Rate limiting is enabled.
- Sensitive fields are removed from logs.
- Dependency and container scanning are performed.

The OWASP API Security Top 10 includes broken authentication, broken authorization, unrestricted resource consumption, unsafe consumption of APIs, and improper inventory management among key API risks. (OWASP Foundation)

Reliability

- Timeouts are configured.
- Retries use exponential backoff.
- Retryable and non-retryable errors are separated.
- Circuit breakers protect failing dependencies.
- Health and readiness endpoints exist.
- Graceful shutdown is supported.
- Queue jobs have retry and dead-letter handling.
- External LLM failures have fallbacks.

Observability

- Structured logs are enabled.
- Request IDs are propagated.
- Metrics and traces are collected.
- p50, p95, and p99 latency are monitored.
- Model, RAG, and agent quality metrics are monitored.
- Alerts exist for errors, latency, drift, cost, and queue growth.
- Dashboards show both technical and business performance.

Scalability

- Load testing has been completed.
- Worker counts are based on memory and CPU/GPU capacity.
- Autoscaling policies are configured.
- Large models use dedicated serving infrastructure.
- Expensive requests have concurrency limits.
- Batch inference is used where real-time responses are unnecessary.

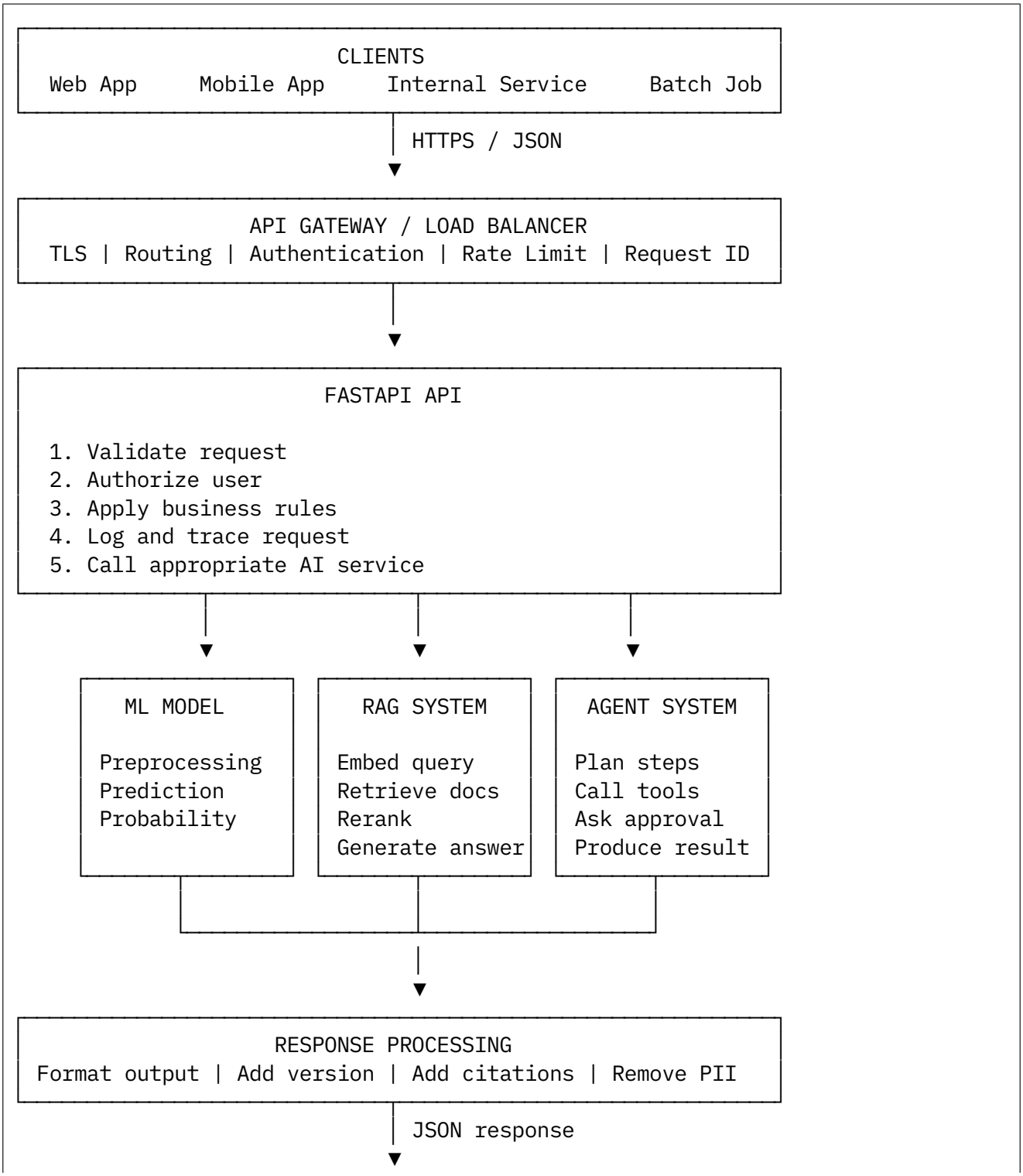
Deployment

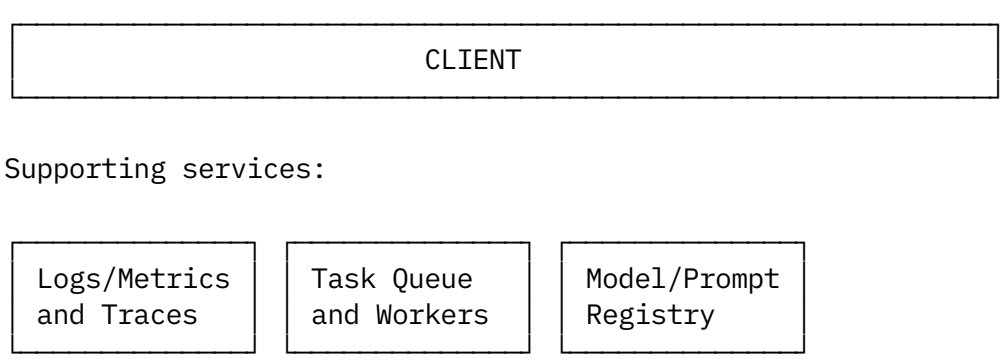
- The application is containerized.
- Development, test, and production configurations are separated.

- CI tests run before deployment.
- Deployment supports rollback.
- Database and vector-index migrations are controlled.
- Canary or blue-green deployment is available.
- Production dependencies are version-pinned.

FastAPI provides deployment guidance for containers, worker processes, HTTPS, and cloud deployment strategies. (FastAPI)

ASCII diagram: Client to API to model flow





Pseudocode for an ML prediction API

```
IMPORT FastAPI, HTTPException, Depends
IMPORT BaseModel, Field
IMPORT logging, timer
IMPORT model_registry
IMPORT authentication_service

CREATE logger

CLASS PredictionRequest:
    customer_id: required string
    age: integer between 18 and 100
    annual_income: positive float
    loan_amount: positive float
    credit_score: integer between 300 and 900

CLASS PredictionResponse:
    request_id: string
    prediction: string
    probability: float
    model_version: string
    processing_time_ms: integer

FUNCTION application_lifespan():
    model = model_registry.load("loan-risk-model", alias="production")
    preprocessor = load_preprocessor(model.version)

    PERFORM warmup_prediction(model)

    STORE model and preprocessor in application state

    START application

    WHEN application stops:
        RELEASE model resources

CREATE FastAPI application using application_lifespan

FUNCTION authenticate_request(token):
    user = authentication_service.validate(token)
```

```
    IF user is invalid:
        RAISE 401 error

    RETURN user

POST "/v1/predictions":

    RECEIVE validated PredictionRequest
    RECEIVE authenticated user

    request_id = generate_request_id()
    start_time = current_time()

    TRY:
        CHECK user has "prediction:execute" permission

        raw_features = convert_request_to_features(request)

        validated_features = verify_training_schema(raw_features)

        transformed_features = preprocessor.transform(validated_features)

        probability = model.predict_probability(transformed_features)

        IF probability >= approval_threshold:
            decision = "approved"
        ELSE:
            decision = "manual_review"

        response = PredictionResponse(
            request_id=request_id,
            prediction=decision,
            probability=probability,
            model_version=model.version,
            processing_time_ms=elapsed_time(start_time)
        )

        LOG:
            request_id
            hashed user identifier
            model version
            latency
            prediction category
            success status

        RECORD prediction metrics

        RETURN response with HTTP 200

    CATCH invalid business data:
        LOG safe validation information
        RETURN HTTP 400 with structured error
```

```
    CATCH model unavailable:
        LOG technical error
        RETURN HTTP 503 with safe message

    CATCH unexpected error:
        LOG complete internal exception
        RETURN HTTP 500 without stack trace
```

Pseudocode for a RAG API

```
CLASS RAGRequest:
    question: string between 3 and 2000 characters
    conversation_id: optional string
    country: optional string
    department: optional string

CLASS Citation:
    document_id: string
    title: string
    page: optional integer
    chunk_id: string

CLASS RAGResponse:
    request_id: string
    answer: optional string
    status: string
    citations: list of Citation
    model_version: string
    retriever_version: string
    processing_time_ms: integer

POST "/v1/rag/query":

    RECEIVE validated RAGRequest
    RECEIVE authenticated user

    request_id = generate_request_id()
    trace = start_trace(request_id)

    TRY:
        CHECK user has "knowledge:query" permission

        normalized_question = normalize(request.question)

        CHECK prompt safety rules
        CHECK question length and language
        CHECK user request quota

        allowed_filters = {
            "security_groups": user.security_groups,
            "country": validate_country_filter(request.country),
            "department": validate_department_filter(
```

```
        request.department,
        user.allowed_departments
    )
}

WITH trace span "embedding":
    query_vector = embedding_model.embed(normalized_question)

WITH trace span "retrieval":
    candidate_chunks = hybrid_search(
        text_query=normalized_question,
        query_vector=query_vector,
        filters=allowed_filters,
        maximum_candidates=50
    )

WITH trace span "reranking":
    ranked_chunks = reranker.rank(
        question=normalized_question,
        chunks=candidate_chunks
    )

selected_chunks = take_top_chunks_within_token_budget(
    ranked_chunks,
    maximum_chunks=5
)

IF selected_chunks contain insufficient evidence:
    RETURN:
        status = "insufficient_evidence"
        answer = null
        citations = []
        safe explanatory message

grounded_prompt = build_prompt(
    system_rules=[
        "Answer only from the supplied context",
        "Do not invent missing information",
        "Cite the source identifiers",
        "Say insufficient evidence when necessary"
    ],
    question=normalized_question,
    context=selected_chunks,
    conversation_history=get_safe_history(
        request.conversation_id,
        user.id
    )
)

WITH trace span "generation":
    generated_answer = llm.generate(
        grounded_prompt,
        maximum_output_tokens=500,
        temperature=0
```

```
        )

    groundedness_result = evaluate_groundedness(
        answer=generated_answer,
        evidence=selected_chunks
    )

    IF groundedness_result fails threshold:
        RETURN safe no-answer response
        RECORD quality failure

    citations = create_citations(
        generated_answer,
        selected_chunks
    )

    STORE audit record:
        request ID
        user authorization scope
        knowledge-base version
        retrieved chunk IDs
        embedding version
        retriever version
        reranker version
        prompt version
        LLM version
        citations
        latency
        token usage

    RETURN RAGResponse:
        answer = generated_answer
        status = "completed"
        citations = citations
        model_version = llm.version
        retriever_version = retriever.version

CATCH vector database timeout:
    RETRY according to policy

    IF retry fails:
        RETURN HTTP 503

CATCH LLM timeout:
    RETURN HTTP 504

CATCH authorization error:
    RETURN HTTP 403

CATCH unexpected error:
    LOG internal exception
    RETURN HTTP 500 with safe message
```

Easy end-to-end web application example

Consider an employee policy assistant.

Frontend

The employee enters:

```
How much paternity leave can I take in India?
```

JavaScript sends:

```
fetch("/v1/rag/query", {
    method: "POST",
    headers: {
        "Content-Type": "application/json",
        "Authorization": "Bearer " + accessToken
    },
    body: JSON.stringify({
        question: userQuestion,
        country: "India"
    })
})
```

Backend

FastAPI:

1. Validates the question.
2. Verifies the employee token.
3. Obtains the employee's document permissions.
4. Searches only authorized policy documents.
5. Reranks the results.
6. Calls the LLM.
7. Validates that the answer is grounded.
8. Returns the answer and citations.

Browser response

```
Employees in India may receive 10 working days of paternity
leave, subject to the eligibility conditions in the policy.

Source: India Parental Leave Policy, page 6.
```

When a browser frontend and backend use different origins, CORS must be configured explicitly rather than allowing every origin without review. FastAPI provides CORS middleware for this purpose. (FastAPI)

Common mistakes

1. Loading the model for every request

```
@app.post("/predict")
def predict(data):
    model = load_model("model.pkl")   # Wrong
```

This increases latency and resource usage. Load the model during application startup.

2. No input validation

Passing client JSON directly to a model can cause incorrect predictions, failures, and security problems.

3. Returning raw exceptions

```
{
  "error": "Database password failure at 10.2.4.8..."
}
```

This exposes infrastructure details.

4. Using `async` for CPU-heavy work and expecting faster inference

`async` helps while waiting for I/O. It does not automatically parallelize CPU or GPU computation.

5. Keeping long agent requests open

A five-minute agent operation should normally return a job ID and run through a worker queue.

6. Logging complete prompts and documents

Prompts may contain personal, confidential, financial, or medical data.

7. Authentication without authorization

A valid employee token must not automatically grant access to every department's documents.

8. Applying RAG permissions after retrieval

Unauthorized documents should never enter the retrieved context.

9. No rate or token limits

An uncontrolled LLM endpoint can create high costs and availability problems.

10. Returning an answer without citations

Users cannot verify whether a RAG answer is based on company information.

11. Not recording model and prompt versions

When a response is challenged, the team cannot identify which system configuration produced it.

12. Too many workers for a large model

Every worker may load another model copy and exhaust memory or GPU resources.

13. No timeout on external calls

An unavailable vector database, LLM, or tool can leave requests hanging.

14. Retrying every error

Retry temporary failures such as timeouts. Do not retry invalid input, denied authorization, or permanent business errors.

15. Mixing all logic inside the API route

Avoid:

```
@app.post("/rag"):
    authenticate()
    query_database()
    embed()
    retrieve()
    rerank()
    build_prompt()
    call_llm()
    evaluate()
    write_logs()
```

Use separate service classes so that each part can be tested independently.

Interview-ready explanation

> We deploy ML and GenAI models behind REST APIs so that web, mobile, and enterprise applications can consume them through a stable interface. I would use FastAPI for request routing, schema validation, authentication, authorization, rate limiting, and response formatting. Small models may be loaded during application startup, while large GPU models should use a separate inference service. RAG endpoints must enforce document-level access control and return citations. Long-running agent workflows should use asynchronous job processing with tool restrictions, human approval, and full audit trails. Production readiness includes structured logging, metrics, traces, timeouts, retries, health checks, versioning, load testing, security scanning, rollback, and continuous monitoring.